

# **Companion to Bird & de Moor, Algebra of Programming**

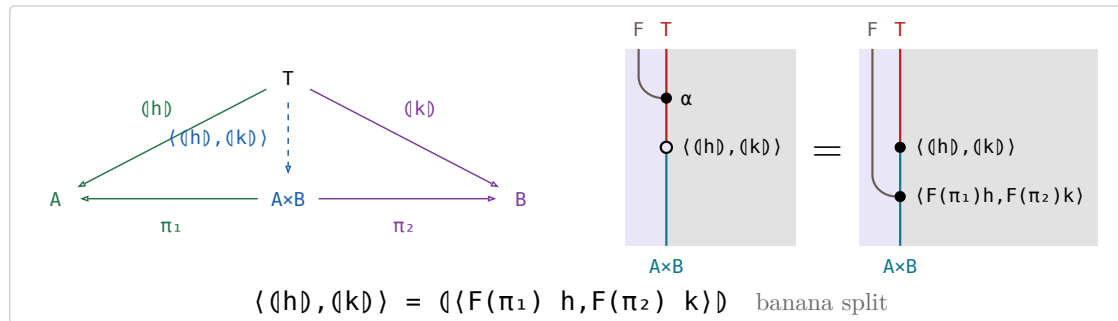
## §3. Applications

### §3.1. Banana-split

(3.1a)

|   | definition                                                                                                                                                                                                   | type                                                    | note                                                                                                         |
|---|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| 1 | $\text{listr } A ::= \text{nil} \mid \text{cons}(A, \text{listr } A) \quad \text{list}(A) = [A]$                                                                                                             | $\mathcal{A} \rightarrow \mathcal{A}$                   | The cons-lists over $A$ , the datatype every row below folds.                                                |
| 2 | $\text{sum} = \llbracket [\text{zero}, \text{plus}] \rrbracket$                                                                                                                                              | $[\mathbb{N}] \rightarrow \mathbb{N}$                   | $\text{plus}(a, b) = a + b$ .                                                                                |
| 3 | $\text{length} = \llbracket [\text{zero}, \pi_2 \text{ succ}] \rrbracket$                                                                                                                                    | $[A] \rightarrow \mathbb{N}$                            | $\pi_2$ drops the head and keeps the count of the tail, $\text{succ}$ adds one for the head.                 |
| 4 | $\text{average} \triangleq \langle \text{sum}, \text{length} \rangle \text{ div}$                                                                                                                            | $[\mathbb{N}] \rightarrow \mathbb{N}$                   | $\text{div}(m, n) = m/n$ , with $\text{div}(0, 0) = 0$ so <b>average</b> is total. Traverses the list twice. |
| 5 | banana-split law<br>$\langle \llbracket h \rrbracket, \llbracket k \rrbracket \rangle = \llbracket (F(\pi_1) h, F(\pi_2) k) \rrbracket$                                                                      | $T \rightarrow A \times B$                              | Any fork of folds is a single fold, hence one traversal — $F$ the base functor.                              |
| 6 | what it reduces to<br>$\alpha \langle \llbracket h \rrbracket, \llbracket k \rrbracket \rangle = F(\langle \llbracket h \rrbracket, \llbracket k \rrbracket \rangle) \langle F(\pi_1) h, F(\pi_2) k \rangle$ | $FT \rightarrow A \times B$                             | All that $\triangleq$ <b>reduce</b> leaves to check: the fork satisfies the defining equation.               |
| 7 | the instance<br>$\langle \text{sum}, \text{length} \rangle = \llbracket [\text{zeros}, \text{pluss}] \rrbracket$                                                                                             | $[\mathbb{N}] \rightarrow \mathbb{N} \times \mathbb{N}$ | $\text{pluss}(a, (b, n)) = (a + b, n + 1)$ , so <b>average</b> runs in one pass.                             |
| 8 | $\text{preds } n = [n, n-1, \dots, 1]$                                                                                                                                                                       | $\mathbb{N} \rightarrow [\mathbb{N}]$                   | apply the earlier special case to write <b>preds</b> as $\llbracket k \rrbracket \pi_1$ .                    |

(3.1b)



### §3.1.1. Fokkinga's mutual recursion theorem

- $F\text{-Alg}(\mathcal{A})$  — the algebras and their homomorphisms — has binary products whenever  $\mathcal{A}$  does, and the forgetful  $U : F\text{-Alg}(\mathcal{A}) \rightarrow \mathcal{A}$  **creates** them: the product of  $h : FA \rightarrow A$  and  $k : FB \rightarrow B$  is carried by  $A \times B$ , with structure  $\langle F(\pi_1)h, F(\pi_2)k \rangle : F(A \times B) \rightarrow A \times B$ .
- $\pi_1$  and  $\pi_2$  are homomorphisms out of it, and  $\langle p, q \rangle : X \rightarrow A \times B$  is a homomorphism **iff**  $p$  and  $q$  both are — substitute the structure and compare the two forks component by component.
- The banana-split law of  $\langle \langle h \rangle, \langle k \rangle \rangle = \langle \langle F(\pi_1)h, F(\pi_2)k \rangle \rangle$  IS that product's universal property read at the initial algebra:  $\langle h \rangle$  and  $\langle k \rangle$  are the unique homomorphisms to  $(A, h)$  and  $(B, k)$ , so their fork is the unique homomorphism into the product, which is  $\langle \langle F(\pi_1)h, F(\pi_2)k \rangle \rangle$ .
- Fokkinga's theorem is **strictly more general** and is not that product: in mutual recursion the two arrows leave  $F(A \times B)$ , not  $FA$  and  $FB$ , so each sees BOTH components. The product is the case where they factor as  $F(\pi_1)h$  and  $F(\pi_2)k$ . What it says: **any algebra on a product carrier is folded by a fork**.

(3.1.1a)



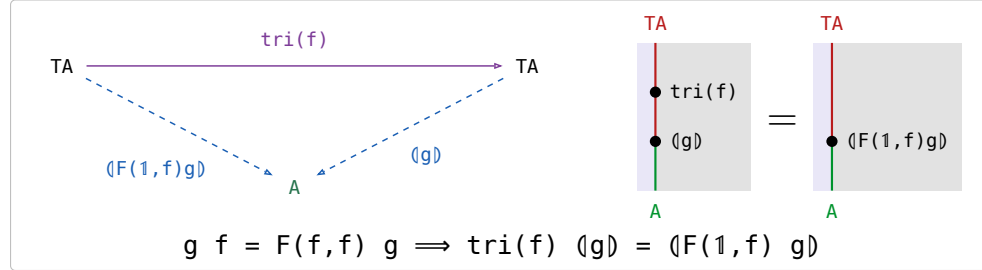
### §3.2. Ruby triangles and Horner's rule

(3.2a)

| stage                         | definition                                                                                                              |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| 1 informally                  | $\text{tri}(f)[a_0, a_1, \dots, a_i, \dots, a_n] = [a_0, f(a_1), \dots, f^i(a_i), \dots, f^n(a_n)]$                     |
| 2 for cons-lists              | $\text{tri}(f) = ([\text{nil}, (1 \times \text{listr}(f)) \text{ cons}])$                                               |
| 3 with the base functor named | $F(A, B) = 1 + A \times B$ , $\alpha = [\text{nil}, \text{cons}]$ , so $\text{tri}(f) = (F(1, \text{listr}(f)) \alpha)$ |
| 4 abstractly                  | $F$ a bifunctor with initial type $(\alpha, T)$ : $\text{tri}(f) = (F(1, T(f)) \alpha)$                                 |

For the definition to make sense  $f : A \rightarrow A$  is required, and then  $\text{tri}(f) : TA \rightarrow TA$ .

(3.2b)



Horner's rule is Horner's rule, generalised from cons-lists to any initial type. Fusion reduces it to  $F(1, T(f)) \alpha (g) = F(1, (g)) F(1, f) g$ .

## §3.2.1. Depth of a tree

(3.2.1a)

|   |                      |                                                                                                                                                                                      |
|---|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | the datatype         | <code>tree A ::= tip(A)   bin(tree A, tree A)</code> , base functor $F(A,B)=A+B \times B$ ,<br>initial type $([tip, bin], tree)$                                                     |
| 2 | the fold             | $\llbracket [g, h] \rrbracket$ is the unique $f$ with $f(tip(a))=g(a)$ and $f(bin(x,y))=h(f(x), f(y))$                                                                               |
| 3 | <code>tree(f)</code> | <code>tree(f) = \llbracket F(f, 1) \rrbracket [tip, bin]</code> ; pointwise <code>tree(f)(tip(a))=tip(f(a))</code> and<br><code>tree(f)(bin(x,y))=bin(tree(f)(x), tree(f)(y))</code> |
| 4 | <code>max</code>     | <code>max = \llbracket [1, bmax] \rrbracket</code> , where <code>bmax(a,b)</code> is the larger of <code>a</code> and <code>b</code>                                                 |
| 5 | <code>depths</code>  | <code>depths = tree(zero) tri(succ)</code> — replaces every tip by its depth in the tree, <code>zero</code> the constant<br>function returning 0 and <code>succ</code> the successor |
| 6 | <code>depth</code>   | <code>depth = depths max</code>                                                                                                                                                      |

### §3.3. The TeX problem

## §3.4. Conditions and conditionals

### §3.5. Concatenation and currying

Theorem 3.1

If  $\varphi$  is natural, i.e.  $G(h \times \text{id}) \cdot \varphi = \varphi \cdot (F(h) \times \text{id})$ , then  $h \cdot G(f) \cdot \varphi = f \iff f = \text{apply} \cdot ((h \cdot G(\text{apply}) \cdot \varphi) \times \text{id})$  The unique solution of a two-argument recursion  $f(a, x) = h(G(f)(a, x))$  built from a natural  $\varphi$  is **apply** composed with a one-argument fold, which is how **cat** — not itself a fold — is derived from one.